Course
Game Design: Theory and Practice

•••

5th Year Computer Science Engineering - Software Engineering

Instructor: Dr. Delloul

Semester 9 / 2026-2027

Chapter 3
Physics and Collisions

Chapter Plan

- Introduction
- Rigid Body Physics
- Physics Engine
- Calculating Collisions
- Physics Engine “Box2DWeb”

Introduction

One of the most important challenges in game development is creating interactions that feel consistent, realistic, and believable. To achieve this, developers need to simulate the behavior of real-world objects using mathematical models and algorithms based on the laws of physics.

These simulations allow objects to move, accelerate, fall under gravity, respond to forces, and interact with one another through collisions. Rather than programming every movement and interaction manually, developers can use physics engines to automate these calculations and simulate physical behavior over time.

Rigid Body Physics

Definition

Rigid body physics is the study of how objects move and interact under the influence of forces, assuming their shape and size do not change.

In game development, rigid body physics help simulate realistic movement, falling objects, bouncing balls, and collisions between objects.

Examples

In a video game, we can have multiple cases such as:

- Objects bouncing;
- Object falling;
- Objects colliding;
- Pushing and pulling objects;
- Shooting objects;
- Objects sliding down...etc



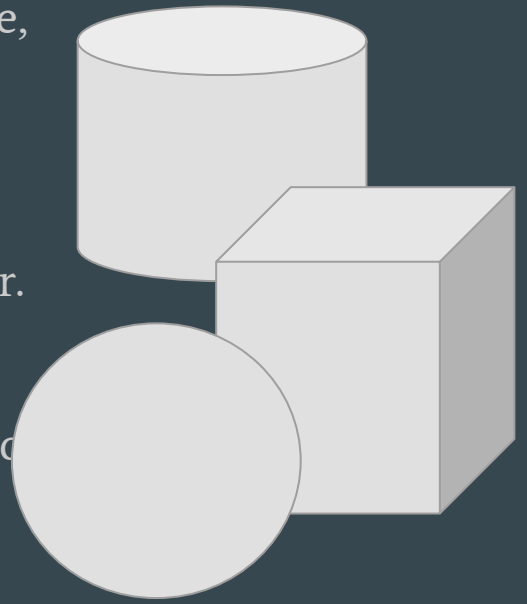
What's a Rigid Body ?

A rigid body is an object that maintains a constant shape and size, even when forces act on it.

A rigid body can move in two fundamental ways:

- **Translation:** The object moves from one position to another.
- **Rotation:** The object turns around an axis or point.

In a typical physics simulation, a rigid body can have both types o motion simultaneously.



Rigid Body Properties

A rigid body has these main properties:

- **Dimensions:** the object's size and shape;
- **Mass:** the amount of matter in an object affects its response to forces;
- **Position:** the location of the object;
- **Velocity:** the speed and direction of its movement;
- **Acceleration:** the rate at which its velocity changes;
- **Gravity:** the effect of gravitational acceleration on its motion;
- **Elasticity:** the way the object behaves during a collision, including how much kinetic energy is retained.

Examples

Translation for Linear Motions

Translation occurs when an object moves without changing its orientation.

$$p_new = p + v * \Delta t$$

where:
p_new: new position
p: position
v: velocity
Δt: is the elapsed time between updates.

Forces and Newton's Second Law

A force is an interaction that can change an object's velocity or direction.

In game development, common forces include:

- Gravity
- Applied forces, such as pushing an object
- Friction
- Air resistance

Newton's second law describes the relationship between force F, mass m, and acceleration a: $F = ma$

Gravity

Gravity is a force that attracts objects toward one another. Near Earth's surface, it causes unsupported objects to accelerate downward.
The gravitational force is:
$F_g = mg$

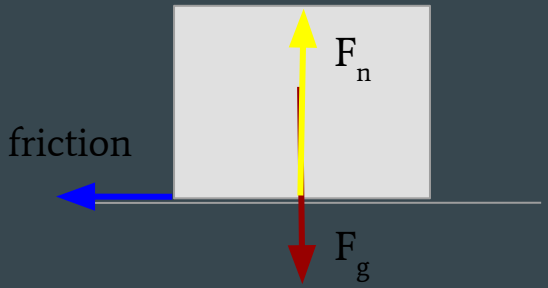
In a game, gravity is often simplified or adjusted to achieve the desired gameplay.

Examples

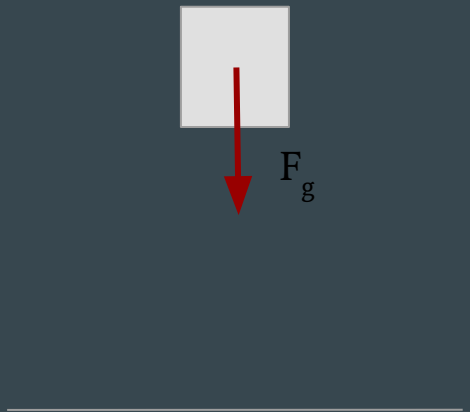
Translation for Linear Motions



Forces and Newton's Second Law



Gravity



Examples

Translation for Linear Motions

Sonic moves from one position to another as he runs across the platform. This is translational motion.



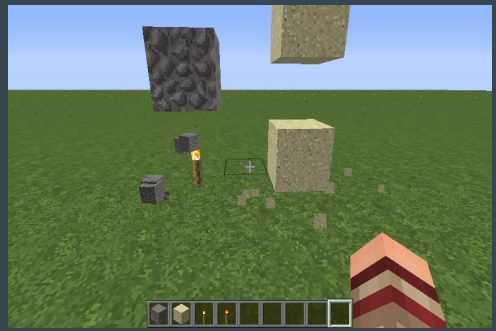
Forces and Newton's Second Law

In Angry Birds, the slingshot launches the bird, and the bird's impact can push blocks and make structures collapse. A net force changes an object's velocity.



Gravity

In Minecraft, when a sand block loses its support, it falls due to gravity until it reaches a surface or another supporting block.



Static, Dynamic, and Kinematic Bodies

1. **Static body**

Does not move under the physics simulation. Commonly used for floors, walls, and fixed platforms.

2. **Dynamic body**

Responds to forces, gravity, and collisions. Commonly used for crates, balls, and movable objects.

3. **Kinematic body**

Its motion is controlled by the game or animation rather than by forces. It can interact with other bodies, depending on the engine's collision rules.

Importance of Rigid Body Physics in Game Development

Without physics simulation, developers might need to manually program:

- How objects fall.
- How objects bounce.
- How moving objects respond to forces.
- How objects rotate when hit.
- How objects stop or slide after contact.

To create such simulations, physics engines were created.

Physics Engine

What is a Physics Engine?

A physics engine is a software library designed to simulate the laws of classical mechanics in a computer environment.

It performs calculations related to physical phenomena such as collisions, gravity, forces, and object motion.

Physics engines are widely used in scientific simulations and video games to create realistic physical interactions.

In game development, a physics engine automates these calculations, determining how objects move and interact over time without requiring developers to program every physical behavior manually.

Example: Instead of manually animating every crate that falls from a platform, a developer can configure its mass, shape, gravity, and collision properties and let the physics simulation calculate its motion.

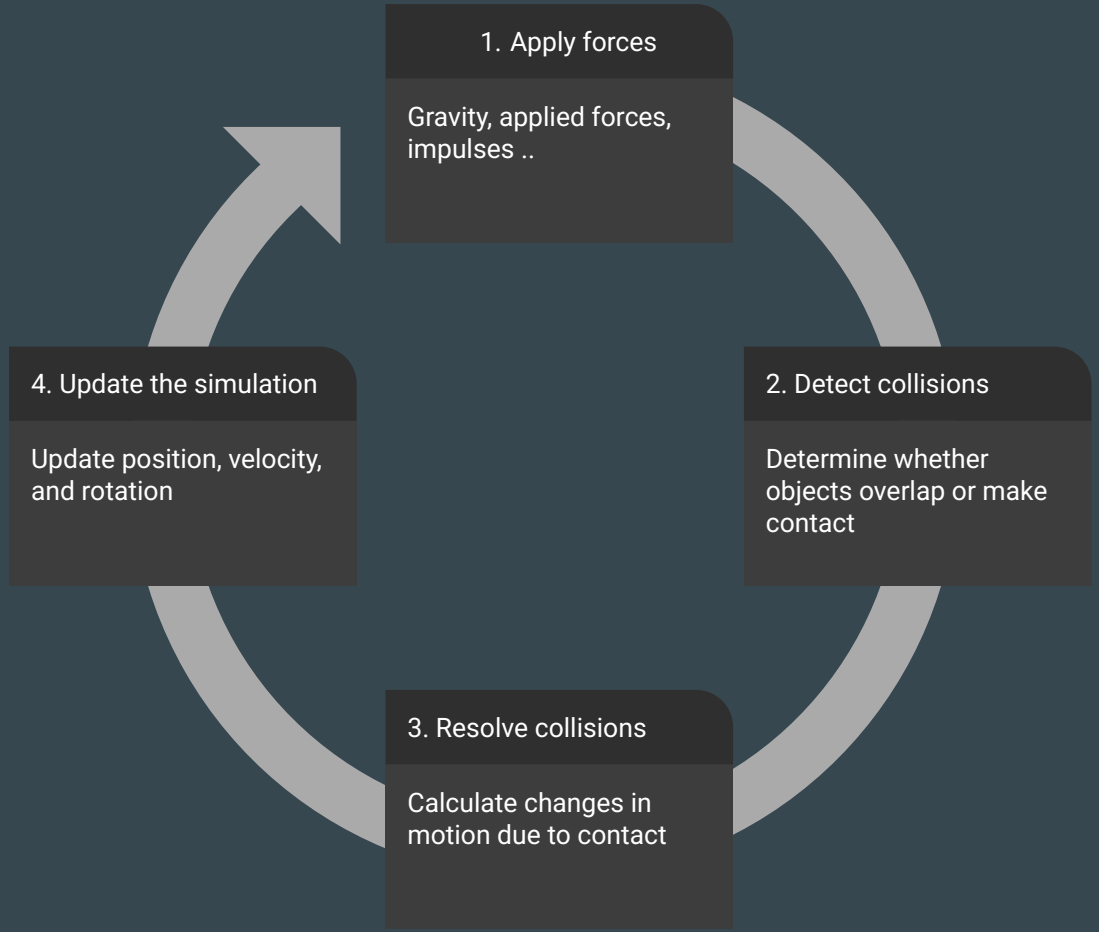
Types of Physics Engines

There are two kinds of physics engines:

1. **2D Physics Engines:**
   Simulate objects in a two-dimensional world, using horizontal and vertical coordinates. Examples: Box2D, Box2DWeb, Matter.js.
2. **3D Physics Engines:**
   Simulate objects in three-dimensional space, including movement and rotation along three axes. Examples: NVIDIA PhysX, Bullet Physics.

Physics Engine Cycle

After defining the objects and their properties (such as mass, velocity, and shape), the physics engine performs a series of calculations at each simulation step to simulate continuous motion. These calculations are repeated in a loop, forming a simulation cycle, illustrated in the diagram alongside.



Main Components of a Physics Engine

The following components work together to simulate physical interactions:

- **Rigid-body simulation:** Calculates object movement and rotation
- **Force and gravity system:** Updates motion based on forces
- **Collision detection:** Identifies objects that touch or overlap
- **Collision response:** Calculates how objects react to contact
- **Constraints and joints:** Connect objects or restrict their movement
- **Physics solver:** Numerically calculates forces and contact responses

Calculating Collisions

What is a Collision ?

A collision occurs when two or more objects come into contact or overlap within a game environment. In video games, collisions are used to determine when objects interact physically, such as when a character touches the ground, a ball hits a wall, or a projectile strikes an enemy.

Collisions allow the game to respond to interactions between objects, making gameplay more realistic and interactive.

Complexity of Collision Detection

In a scene containing n objects, each object may need to be checked against every other object to determine whether a collision occurs. This gives a time complexity of $O(n^2)$.

Why is this expensive?

- Each object may collide with multiple other objects.
- The number of possible pairs grows quadratically.
- Checking every pair becomes costly in scenes with many objects.

So it is necessary to optimize the search for colliding objects.

Optimization: Two-Phase Approach

The two-phase approach consists of starting by filtering potential collisions, then performing precise tests only on the remaining pairs.



All Objects

Potential collisions

Broad Phase

Quickly identifies potentially colliding pairs

Narrow Phase

Precisely tests the candidate pairs

Collision Information

Gets information necessary to updating the scene

Broad Phase: Reducing Collision Checks

This method:

- Uses simplified bounding volumes, such as AABBs, circles, and spheres.
- Eliminates pairs that are too far apart to collide.
- Can use spatial grids, quadtrees, BVHs, or Sweep and Prune.

➔ Goal: reduce the number of pairs passed to the next phase.

The broad phase eliminates many pairs that cannot collide, significantly reducing the number of expensive geometric tests in many scenes. However, its performance depends on the scene and the algorithm used; the worst-case complexity can still be $O(n^2)$.

Broad Phase: Bounding Boxes Types

A bounding box is a simple geometric shape that completely encloses an object, used to simplify and speed up collision detection by checking whether objects overlap.

In the broad phase, there are many types of bounding boxes that can be used:

1. **AABB (Axis-Aligned Bounding Box):** A box aligned with the coordinate axes.
2. **OBB (Oriented Bounding Box):** A box that can rotate with the object.
3. **Circles:** Commonly used for 2D collision detection.
4. **Spheres:** Commonly used for 3D collision detection.

Broad Phase: Bounding Boxes Types



Examples of bounding boxes

Broad Phase: Algorithms

An example of bounding box of type AABB calculation:

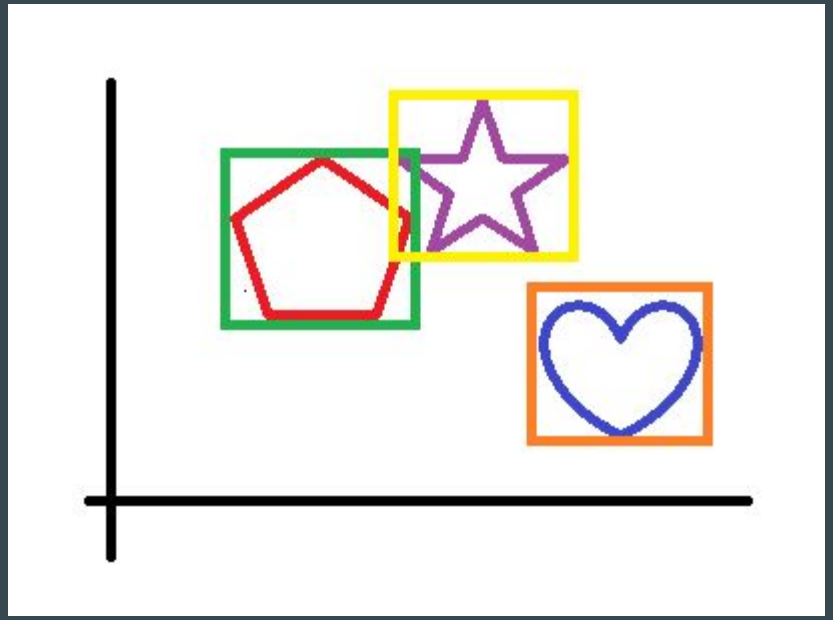
```
Algorithm CalculateAABB(vertices):
Start
 xMin ← vertices[0].x;  xMax   ← vertices[0].x; yMin   ← vertices[0].y;  yMax   ← vertices[0].y
For each vertex in vertices do
   If vertex.x < xMin then xMin    ← vertex.x;
  End If;
   If vertex.x > xMax then xMax    ← vertex.x;
   End If;
   If vertex.y < yMin then  yMin    ← vertex.y;
   End If;
   If vertex.y > yMax then  yMax    ← vertex.y;
   End If;
 End For;
Return (xMin, yMin), (xMax, yMax)
End Algorithm.
```

Broad Phase: Algorithms

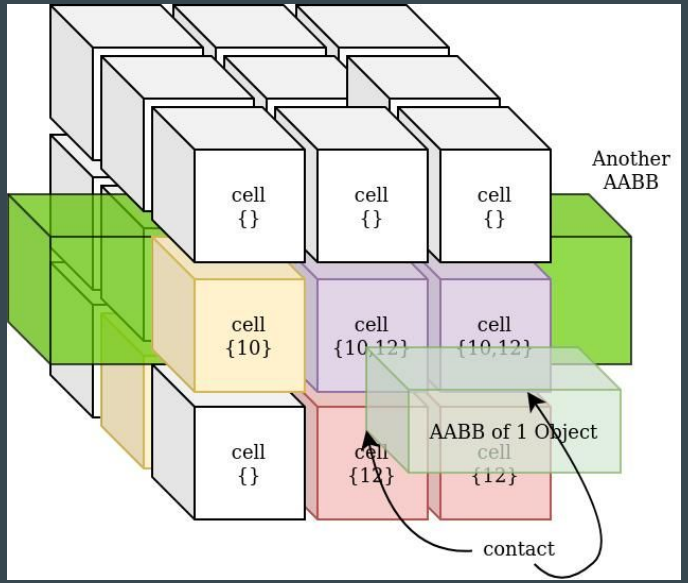
So if we base the collision detection on AABB bounding boxes, it means we need to find overlapping boxes.

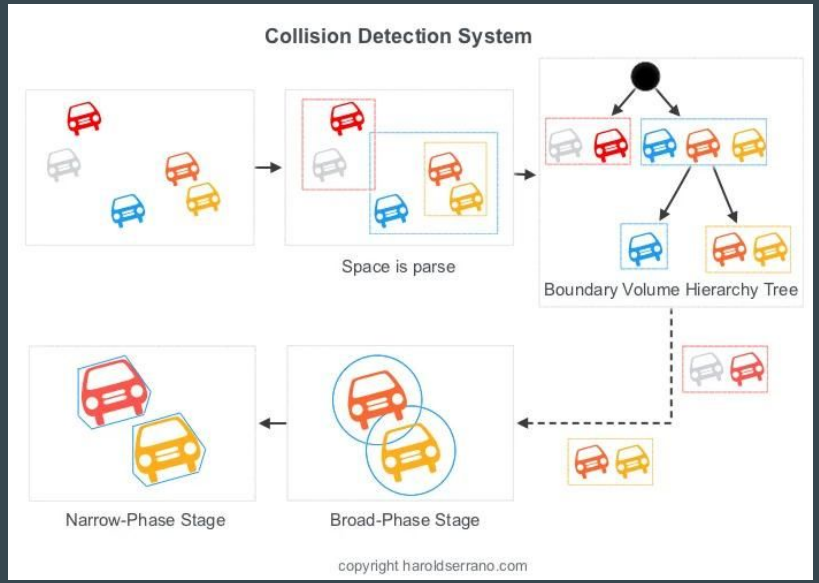
To reduce the computational complexity of collision detection, the search space can be reduced by partitioning it using techniques such as spatial grids, quadtrees, BVHs, or Sweep and Prune.

One of the most widely used algorithms in physics engines is Sweep and Prune (Sort and Sweep).



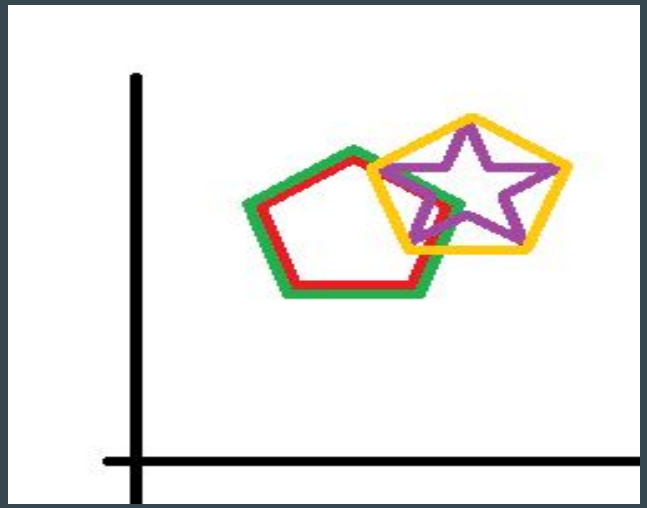
Broad Phase: Algorithms





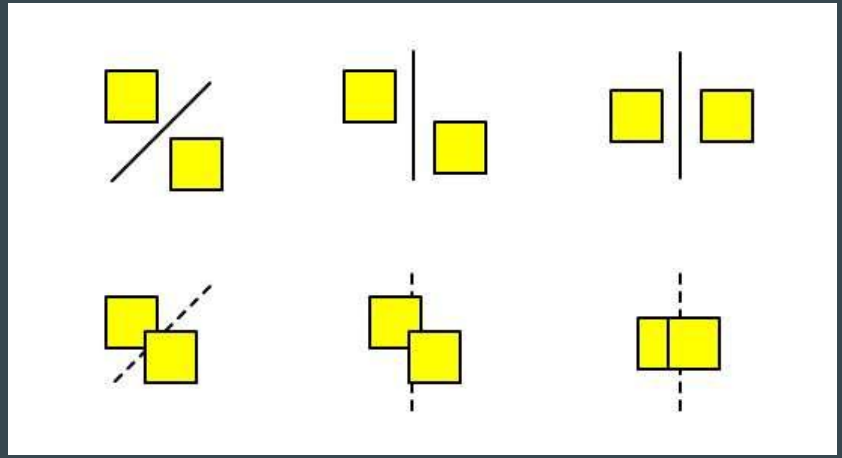
Narrow Phase

- Examines candidate pairs from the broad phase.
- Uses more precise geometric tests to determine whether objects actually collide.
- Can calculate contact points, contact normals, and penetration depth.



Narrow Phase: Algorithms

- Separating Axis Theorem (SAT) stating that “If two convex objects are not penetrating, there exists an axis for which the projection of the objects will not overlap.”
- GJK (Gilbert–Johnson–Keerthi) Algorithm: An iterative algorithm used to detect collisions and calculate the distance between two convex shapes.



Physics Engines and Collision Detection

When an object moves very fast in a game, Continuous Collision Detection (CCD) is used by physics engines to detect collisions so the object doesn't accidentally pass through walls or other objects without hitting them.

Example: A bullet moving very fast toward a wall.

- Without CCD: The bullet might pass through the wall because the game checks collisions only at certain moments.
- With CCD: The game checks the bullet's movement between those moments and detects that it hit the wall.

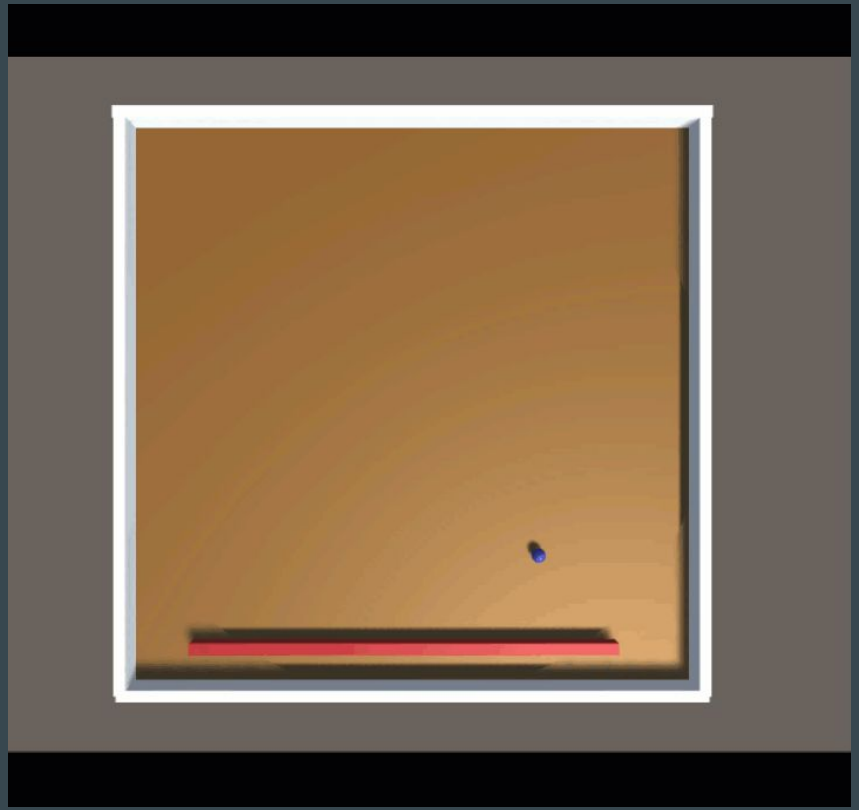
This problem is called **tunneling** — when a fast-moving object passes through another object without a collision being detected.

Physics Engines and Collision Detection

The key difference:

- Discrete collision detection: Checks positions at specific time steps.
- Continuous collision detection: Checks movement over the interval between time steps.

So, CCD doesn't necessarily test more often or every frame. Instead, it considers the path an object travels between physics updates to detect collisions that ordinary checks might miss.



Physics Engine: Box2DWeb

Physics Engine “Box2D”

Box2D is a 2D rigid body simulation library for games. Programmers can use it in their games to make objects move in realistic ways and make the game world more interactive. From the game engine's point of view, a physics engine is a system for procedural animation.

https://box2d.org/documentation/

“Box2D” in Games



Limbo



Angry Birds



Shovel Knight

“Box2DWeb”

Originally, Box2D was written in C++. To use its physics engine in a browser-based JavaScript game, developers created ports such as Box2DWeb. Box2DWeb is specifically based on the older Box2DFlash 2.1a version, rather than being a port of every version of Box2D.

“Box2DWeb”: Elements

1. **Shape:**

A geometric form that defines the physical geometry of an object in 2D space.

- Examples: circles, rectangles, and polygons.
- Defines the object's collision boundaries.
- A shape alone does not have physical properties such as mass or friction.

“Box2DWeb”: Elements

2. Fixture:

A component that attaches a shape to a rigid body and defines its physical and collision properties.

- Density: Determines how much mass the shape contributes based on its area.
- Friction: Controls resistance when surfaces slide against each other.
- Restitution: Determines how bouncy an object is during collisions.
- Collision filtering: Controls which objects can collide with each other.

“Box2DWeb”: Elements

3. Rigid Body:

A physical object in the simulation that has a position, rotation, velocity, and other physical properties.

- Can contain one or more fixtures.
- Can move, rotate, and collide with other bodies.
- Common body types: static, dynamic, kinematic.

“Box2DWeb”: Elements

4. World:

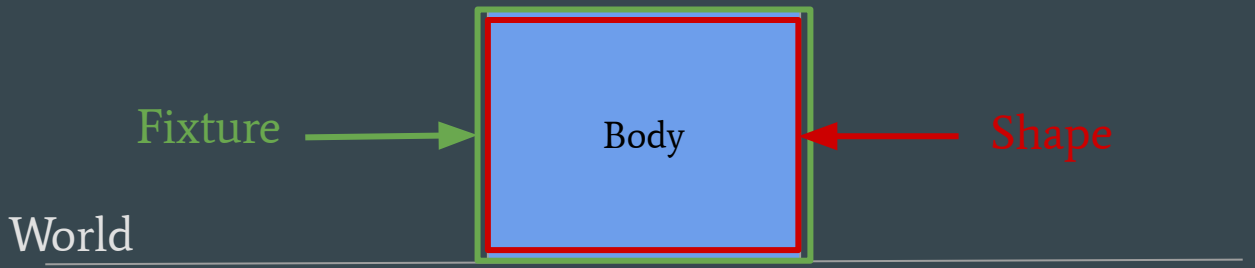
The 2D physics environment that contains bodies and simulates their interactions.

- Manages gravity.
- Calculates collisions and physical responses.
- Updates the positions and velocities of bodies over time.
- Can contain all types of bodies.

“Box2DWeb”: How Elements Work Together

World → Rigid Body → Fixture → Shape

For example, a falling box in a game consists of a dynamic rigid body, a rectangular shape attached through a fixture, and a world with gravity enabled.



“Box2DWeb”

The essential steps:

1. Create the world.
2. Create bodies and add them to the world.
3. Simulate the world.

“Box2DWeb”: World Creation

Use the classes *Vec2* and *World* to build a world with its gravity:

```
var Vec2 = Box2D.Common.Math.b2Vec2;
var World = Box2D.Dynamics.b2World;
// horizontal and vertical gravity
var gravity = new Vec2(0, 9);
// create world
var world = new World(gravity, true);
```

“Box2DWeb”: Body Creation

To create a body we need the following classes:

```
var BodyDef = Box2D.Dynamics.b2BodyDef;
var Body = Box2D.Dynamics.b2Body;
var FixtureDef = Box2D.Dynamics.b2FixtureDef;
var Polygon = Box2D.Collision.Shapes.b2PolygonShape;
```

“Box2DWeb”: Body Creation

We define a fixture:

```
var fixDef = new FixtureDef();
// density
fixDef.density = 1;
// friction
fixDef.friction = 0.4;
// restitution : for rebound after collision
// 0 = no rebound ; 1 = rebound
fixDef.restitution = 0.2;
```

“Box2DWeb”: Body Creation

Create a body and define its shape, for example ground:

```
var groundDef = new BodyDef();
// the ground won't move due to forces so it's static
groundDef.type = Body.b2_staticBody;
// position of the center of the ground, from pixels to MTS
groundDef.position.Set(400 / scale, 280 / scale);
// give a rectangular "shape" to the ground
var groundShape = new Polygon();
// SetAsBox takes half-width and half-height of the body
groundShape.SetAsBox(200 / scale, 20 / scale);
```

“Box2DWeb”: Body Creation

For a body we have to associate the shape to the fixture then the fixture to the body and add the body to the world:

```
// associate the shape to the fixture
fixDef.shape = groundShape;
// add body to the world
// don’t forget to associate fixture to the body
world.CreateBody(groundDef).CreateFixture(fixDef);
```

“Box2DWeb”: Body Creation

We can change the type of the created body using the BodyDef.type:

```
// for dynamic bodies
bodyDef.type = b2Body.b2_dynamicBody;

// for kinematic bodies
bodyDef.type = b2Body.b2_kinematicBody;
```

“Box2DWeb”: Visualization

To visualize, we use the *DebugDraw* class :

```
var DebugDraw = Box2D.Dynamics.b2DebugDraw;
// tool that draws on the screen
var debug = new DebugDraw();
// precise on which context it's going to draw
debug.SetSprite(context);
// convert coordinates to pixels
debug.SetDrawScale(scale);
// associate the drawing tool to the world
world.SetDebugDraw(debug);
// draw the bodies of this world, once (one call)
world.DrawDebugData();
```

“Box2DWeb”: Scale ?

Box2D uses the MTS (Meter–Tonne–Second) system of units to represent physical quantities. Distances are expressed in meters, mass in tonnes, and time in seconds. This system is suitable for simulating large environments and objects.

Before drawing objects on the HTML canvas, we must convert between Box2D world coordinates (meters) and canvas coordinates (pixels).

$$scale = pixels / meter$$

Example: scale = 20 means 20 pixels represent 1 meter

“Box2DWeb”

Important:

- In Box2D, objects are positioned using their center as the reference point. The SetAsBox() method takes half the width and half the height of the rectangle as arguments, rather than its full dimensions.
- The scale factor is a conversion chosen by the programmer, not a fixed Box2D value.
- To animate, we need to use a gameloop in this case as well.

“Box2DWeb”: Collision Detection

Box2D uses two main phases:

1. Broad phase:
   It quickly identifies pairs of objects that might collide, using their bounding boxes (AABBs).
2. Narrow phase:
   It performs more precise geometric tests to determine whether the candidate objects actually collide.

“Box2DWeb”: Collision Detection

Once a collision is detected, Box2D calculates the appropriate physical response.

The response depends on several properties:

Density	Determines mass through the shape’s volume/area
Friction	Controls resistance to sliding
Restitution	Controls bouncing
Body Type	Determines how a body reacts to forces and collisions

“Box2DWeb”: Collision Detection

Box2D provides a contact listener that allows us to execute JavaScript code when two bodies begin or finish touching.

First, create a contact listener:

```
var listener = new Box2D.Dynamics.b2ContactListener();
```

“Box2DWeb”: Collision Detection

Then define the callback functions:

```
// Called when two fixtures start touching
listener.BeginContact = function(contact) {
    console.log("Collision started!");
};
// Called when two fixtures stop touching
listener.EndContact = function(contact) {
    console.log("Collision ended!");
};
```

“Box2DWeb”: Collision Detection

Finally, register the listener with the world:

```
world.SetContactListener(listener);
```

Now Box2D will notify your program when contacts begin and end.

“Box2DWeb”: Collision Detection

Box2D also provides other useful methods:

BeginContact()	Called when two fixtures begin touching
EndContact()	Called when two fixtures stop touching
PreSolve()	Called before the collision response is calculated
PostSolve()	Called after the collision response is calculated

These methods allow us to customize collision behavior.

“Box2DWeb”: Collision Response

After detecting the collision, Box2D's physics engine automatically calculates how the objects react physically (stopping, bouncing, sliding, etc.) which is the process of generating a response to detected collision.

In addition, we can define other behaviors to occur when a collision is detected, depending on the needs of the application. For example, we can increment a score, count the number of collisions between two objects, change an object's color, display a message, play a sound, or trigger a specific event. These actions are implemented by the programmer using the collision callbacks (`BeginContact` and `EndContact`).

“Box2DWeb”: Collision Response

Example:

```
// Load the collision sound
var collisionSound = new Audio("collision.mp3");

listener.BeginContact = function(contact) {
    console.log("Collision started!");
    // Play the collision sound
    collisionSound.currentTime = 0;
    collisionSound.play();
};

listener.EndContact = function(contact) {
    console.log("Collision ended!");
};

world.SetContactListener(listener);
```

References

- MDN Web Docs — 3D Collision Detection
- Box2D Official Documentation
- MDN Web Docs — Game Development Techniques
- Physics for Game Developers. David M. Bourg and Bryan Bywalec · 2nd edition · 2013
- Game Physics Cookbook. Gabor Szauer · Packt Publishing · 2017
- NVIDIA Developer. Chapter 32. Broad-Phase Collision Detection with CUDA
- Unity Manual